

Multiple Programming Paradigms — Define Paradigms — Answers

Q1. Define procedural, OOP, and functional paradigms with Java examples.

Procedural paradigm:

A programming approach in which a program is organized as a sequence of instructions and procedures (methods/functions) that operate on data, typically using global or shared variables and step-by-step logic.

```
public class ProceduralExample {  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int result = add(5, 10);  
        System.out.println("Sum: " + result);  
    }  
}
```

Object-oriented paradigm (OOP):

A programming approach that organizes a program around objects, which bundle data (fields) and behavior (methods) together, supported by concepts such as encapsulation, inheritance, and polymorphism.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class OopExample {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println("Sum: " + calc.add(5, 10));  
    }  
}
```

Functional paradigm:

A programming approach that treats computation as the evaluation of functions, favoring immutability and functions as first-class values rather than changing shared state. Java supports this style through lambda expressions and the functional interfaces introduced in

Java 8.

```
import java.util.function.BinaryOperator;

public class FunctionalExample {
    public static void main(String[] args) {
        BinaryOperator<Integer> add = (a, b) -> a + b;
        System.out.println("Sum: " + add.apply(5, 10));
    }
}
```

Q2. Write the same logic using both procedural and object-oriented approaches. Consider the logic for calculating the area of a rectangle.

Procedural approach:

```
public class RectangleProcedural {
    static double calculateArea(double length, double width) {
        return length * width;
    }

    public static void main(String[] args) {
        double length = 10, width = 5;
        double area = calculateArea(length, width);
        System.out.println("Area: " + area);
    }
}
```

Object-oriented approach:

```
class Rectangle {
    private double length;
    private double width;

    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    double calculateArea() {
        return length * width;
    }
}

public class RectangleOop {
```

```

    public static void main(String[] args) {
        Rectangle rect = new Rectangle(10, 5);
        System.out.println("Area: " + rect.calculateArea());
    }
}

```

Both versions compute the same result. In the procedural version, length and width are passed as separate arguments to a standalone method. In the object-oriented version, length and width are bundled together with the calculateArea behavior inside a single Rectangle object, which better models the real-world entity and keeps its data and behavior together.

Q3. Identify Java features that support more than one paradigm.

Although Java is primarily known as an object-oriented language, several of its features allow it to support procedural and functional styles as well, making it a multi-paradigm language:

- Static methods: methods declared static can be called without creating an object, allowing a procedural, function-call style of programming.
- Classes, interfaces, inheritance, and polymorphism: these support the object-oriented paradigm by organizing code around objects and their relationships.
- Lambda expressions (introduced in Java 8): allow behavior to be passed around as values, supporting a functional style of programming.
- Functional interfaces (such as Function, Predicate, Supplier, and BinaryOperator): provide a foundation for treating functions as first-class citizens.
- The Stream API: enables functional-style operations such as map, filter, and reduce on collections of data, without explicit loops.

This combination of features is what allows the same Java program to freely mix procedural logic, object-oriented design, and functional-style expressions.